



Instituto Politécnico
Escuela Superior de Cómputo
ESCOM



Carrera:

Ingeniería en Sistemas Computacionales

Academia

Sistemas Digitales

Unidad de aprendizaje:

Sistemas embebidos

Nombre Tarea

Proyecto Integrador: Sistema IoT de
Monitoreo de Cadena de Frío mediante
MQTT

Alumno

Rodríguez Cejudo Marlon Francois
2022630549

Profesor

Miguel Ángel Aleman Arce

Contenido

Introducción	3
Productos esperados	4
Resultados esperados	4
Justificación	5
Desarrollo	7
Prueba unitaria sensor BMP280	7
Prueba unitaria sensor DS18B20	9
Prueba unitaria sensor MC-38	11
Integración de sensores y comunicación MCU-MPU (Arduino Uno Q)	13
Lectura de los 3 sensores	14
Bridge / RPC	14
Contrato de comunicación MCU-MPU	14
Responsabilidades del MCU	15
Responsabilidades del MPU	15
Modelo de comunicación	15
Servicio RPC propuesto	15
La solicitud del MPU al MCU	16
Respuesta del MCU al MPU	16
Manejo de errores	19
Datos agregados por el MPU	20
Reglas de estado del sistema	21
Frecuencia de lectura	21
Tópicos MQTT asociados	21
Diagrama general de comunicación entre MCU y MPU	21
Programa MCU con cache	22
Programa MPU	28
Instalación Mosquitto en la UNO Q	29
Funcionamiento	33

Conclusiones	34
--------------------	----

Introducción

La preservación de la integridad de productos termosensibles, tales como medicamentos, vacunas y alimentos perecederos, depende estrictamente del mantenimiento ininterrumpido de la cadena de frío. Cualquier fluctuación fuera de los rangos térmicos estipulados o la apertura descontrolada de los contenedores de transporte puede alterar las propiedades de estos bienes, provocando mermas económicas significativas y, en el peor de los casos, graves riesgos para la salud pública. Históricamente, la supervisión de estas variables se realizaba de forma pasiva o manual, lo que impedía una capacidad de respuesta en tiempo real ante incidentes.

Romper la cadena de frío ocurre cuando la mercancía sale del rango de temperatura exigido por la normativa sanitaria. Es mucho más que un cambio puntual. La continuidad es el factor más importante para garantizar la seguridad alimentaria. Los peligros en la rotura de los niveles térmicos generan riesgos críticos. El impacto no siempre es visible a simple vista. Afecta tanto a la seguridad del consumidor como a la viabilidad comercial del producto, algunos de los peligros son:

- **Proliferación de bacterias:** El calor activa el crecimiento de microorganismos peligrosos. Bacterias como la Salmonella o la Listeria se reproducen rápido a temperaturas inadecuadas. Esto convierte un alimento sano en un producto tóxico en poco tiempo.
- **Pérdida de valor nutricional:** Las vitaminas y proteínas son sensibles al calor. Cuando la temperatura sube, estos componentes se degradan. El producto pierde sus propiedades beneficiosas originales. El consumidor recibe un alimento menos nutritivo de lo esperado.
- **Aparición de toxinas:** Algunas bacterias generan sustancias tóxicas cuando se calientan. Estas toxinas pueden resistir incluso el cocinado posterior. Es uno de los mayores peligros porque no se eliminan con el calor del fuego o del horno.
- **Alteración de la calidad organoléptica:** El sabor y la textura cambian drásticamente. El color puede volverse extraño y el olor desagradable. Estas señales indican que el producto ya no es apto para su venta o consumo.

Todas estas consecuencias en la ruptura de la cadena de frío tienen consecuencias directas en las finanzas, se pierde el producto y la empresa tiene que asumir los costes operativos, incluso legales según el tipo de producto que manipule.

En este contexto, el Internet de las Cosas (IoT) surge como una tecnología disruptiva capaz de transformar la logística de distribución. Este proyecto presenta el diseño e implementación de un sistema de monitoreo automatizado en tiempo real para la cadena

de frío. Utilizando una arquitectura basada en una placa de desarrollo híbrida de computadora de placa única y microcontrolador (UNO Q) y el protocolo de comunicación liviano MQTT, el sistema adquiere datos ambientales y de estado del contenedor a través de sensores especializados de temperatura, presión y apertura magnética. Esta información es procesada y transmitida de forma jerárquica hacia un bróker Mosquitto, permitiendo la activación de alertas tempranas y la visualización remota de las variables críticas para mitigar proactivamente las rupturas en la cadena de distribución.

Productos esperados

El desarrollo de este proyecto contempla la entrega de un sistema tecnológico integral y funcional, compuesto por elementos de hardware y soluciones de software interconectadas. En primer lugar, se presentará un prototipo físico de nodo publicador, el cual estará integrado por una placa de desarrollo Arduino UNO Q y los sensores DS18B20, BMP280 y el interruptor magnético MC-38. Además de un ESP32 el cual estará suscrito a los eventos y este ejecutará los actuadores, como lo son una luz de emergencia y una alarma. Vinculado a este hardware, se entregará el firmware optimizado, programado para gestionar de forma eficiente la lectura de las variables y asegurar la estabilidad de la conexión a la red.

En el ámbito de la comunicación y el procesamiento, el proyecto entregará una instancia del bróker Mosquitto MQTT completamente configurada, cuya arquitectura se basará en una estructura de tópicos jerárquicos diseñada para garantizar un flujo de datos ordenado y escalable. Finalmente, se dispondrá de un nodo suscriptor que se materializará en una interfaz de usuario o panel de control, la cual incluirá un módulo lógico programado para procesar la información entrante, desplegar las métricas en tiempo real y ejecutar las acciones correspondientes a la activación de alertas o actuadores periféricos.

Resultados esperados

Con la implementación de los productos mencionados, se prevé alcanzar una serie de impactos significativos en la gestión y seguridad de la logística de frío. El resultado primordial será la obtención de una trazabilidad térmica continua y de alta precisión, logrando aislar la temperatura interna del producto de las variaciones climáticas del entorno gracias al análisis comparativo de los datos recolectados. Asimismo, el sistema permitirá la correlación directa de eventos críticos, facilitando la identificación inmediata

de las causas detrás de una anomalía, como podría ser una elevación de la temperatura provocada por la apertura prolongada de las compuertas del contenedor.

Por otra parte, se espera una optimización sustancial en los tiempos de respuesta operativa. La automatización de las notificaciones mediante el protocolo MQTT garantizará que el personal encargado reciba alertas instantáneas ante cualquier desviación de los umbrales de seguridad establecidos, transformando la gestión de la cadena de suministro de un enfoque reactivo a uno preventivo. Por último, desde una perspectiva técnica y económica, se validará la eficiencia del uso de arquitecturas IoT de bajo costo y consumo ligero de ancho de banda para la mitigación efectiva del desperdicio de bienes sensibles, asegurando la conservación de su calidad y reduciendo las pérdidas materiales en el sector alimentario o farmacéutico.

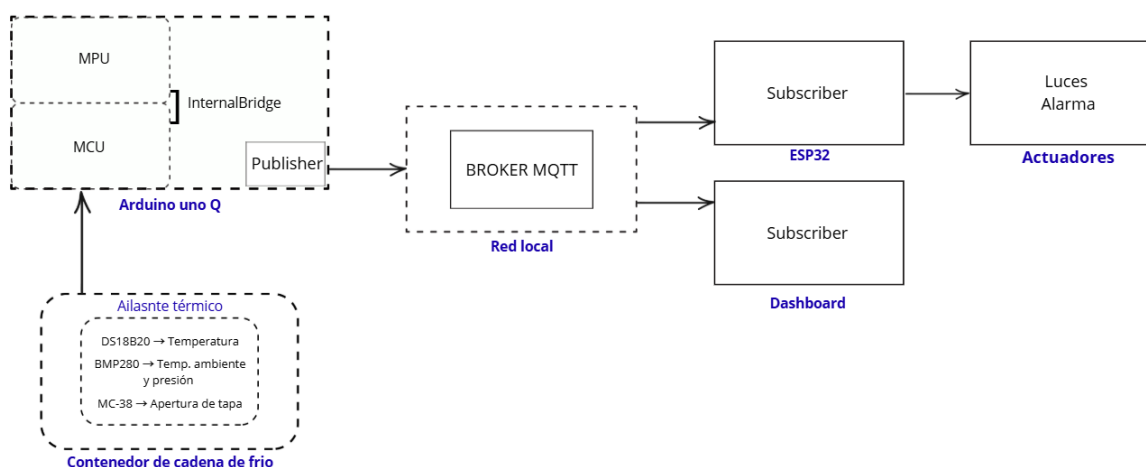


Figura 1. Arquitectura del Sistema

Justificación

La correcta preservación de productos termosensibles durante la fase de distribución representa uno de los mayores desafíos logísticos y de salud pública en la actualidad. Tanto en la industria alimentaria como en la farmacéutica, las rupturas en la cadena de frío no solo se traducen en pérdidas económicas multimillonarias por el desperdicio de mercancías, sino que también ponen en riesgo directo a los consumidores finales al acelerar la descomposición de los alimentos o neutralizar la efectividad de medicamentos y vacunas esenciales. Tradicionalmente, el control de estos entornos se ha limitado a registros históricos pasivos que se analizan únicamente al concluir el trayecto, un enfoque

reactivo que impide intervenir a tiempo para salvar la carga ante un fallo en los sistemas de refrigeración o una manipulación inadecuada de los contenedores.

Frente a esta problemática, el desarrollo de una solución basada en el Internet de las Cosas (IoT) se justifica por la necesidad imperativa de transitar hacia un modelo de supervisión proactivo y en tiempo real. La integración de hardware accesible como la plataforma Arduino UNO Q, en conjunto con sensores especializados (DS18B20 y BMP280), demuestra que es técnicamente viable y económicamente rentable implementar telemetría de alta precisión sin incurrir en los elevados costos de los sistemas industriales cerrados. Asimismo, la incorporación del sensor magnético MC-38 añade una capa crítica de seguridad operativa, permitiendo correlacionar de forma inmediata el comportamiento de la temperatura con las aperturas físicas del contenedor, aislando así las causas humanas de las fallas mecánicas.

Finalmente, la adopción del protocolo MQTT sobre un bróker Mosquitto proporciona la eficiencia necesaria para entornos de movilidad logística, donde el ancho de banda suele ser limitado o inestable. Al organizar la información mediante tópicos jerárquicos y transmitir datos únicamente cuando es necesario o bajo un esquema liviano de publicación y suscripción, se garantiza un consumo mínimo de recursos de red y una latencia casi nula en la emisión de alertas. En consecuencia, este proyecto se justifica plenamente al ofrecer una herramienta tecnológica escalable y de bajo costo que democratiza el acceso al monitoreo inteligente, reduce el desperdicio de bienes críticos y asegura los estándares de calidad exigidos por el mercado y las normativas sanitarias vigentes.

Desarrollo

Con el propósito de garantizar la fiabilidad del sistema antes de su integración general, se procedió a la ejecución de pruebas unitarias exhaustivas con cada uno de los sensores de manera aislada utilizando la placa Arduino UNO Q. Este proceso de validación individual permitió verificar la correcta comunicación entre el microcontrolador y los periféricos, así como calibrar las lecturas bajo entornos controlados. De este modo, se comprobó de forma independiente la precisión en la captura de datos térmicos del sensor DS18B20, la estabilidad de las mediciones de presión y temperatura ambiental del BMP280, y la respuesta inmediata de conmutación del sensor magnético MC-38. Estas pruebas unitarias aseguraron que cada componente de hardware operara de acuerdo con las especificaciones del fabricante, mitigando la existencia de fallas por ruido eléctrico o errores de direccionamiento en las etapas posteriores del desarrollo.

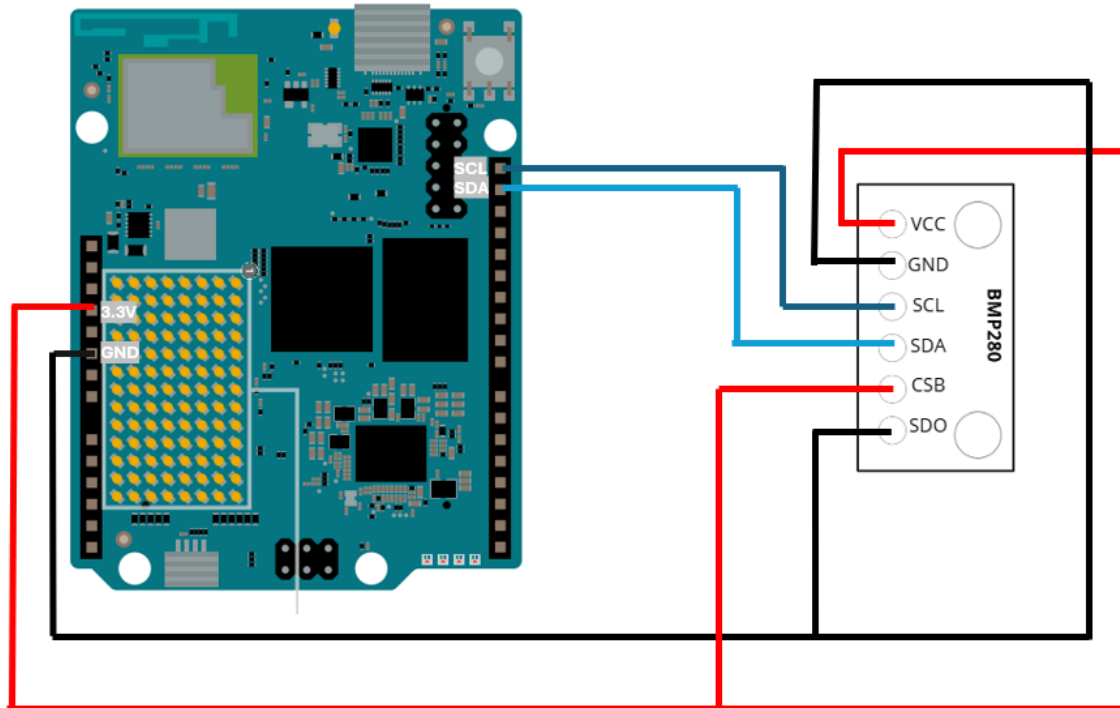
Prueba unitaria sensor BMP280

Para esta prueba tengo instalado el IDE de Arduino (clásico), en el cual se deben descargar las siguientes librerías

- Adafruit BMP280 Library
- Adafruit Unified Sensor
- Adafruit BusIO

Una vez que se tienen las librerías, se agrega el código correspondiente del sensor, el cual se puede consultar en el repositorio.

Además, para la conexión del sensor y del Arduino Uno Q, es el siguiente.



Conexión Arduino Uno Q con sensor BMP280

Después de tener la conexión lista, compilamos y ejecutamos el código y se observan la siguiente imagen.

```
1 #include <Arduino_RouterBridge.h>
2 #include <Wire.h>
3 #include <Adafruit_BMP280.h>
4
5 Adafruit_BMP280 bmp;
6
7 float tempAnterior = 0;
8 float presAnterior = 0;
9
10 void setup() {
11   Monitor.begin();
12   delay(2000);
13
14   Monitor.println("Prueba BMP280 - medicion forzada");
15   Monitor.println("Usando pines SDA/SCL dedicados del Arduino UNO Q");
16 }
```

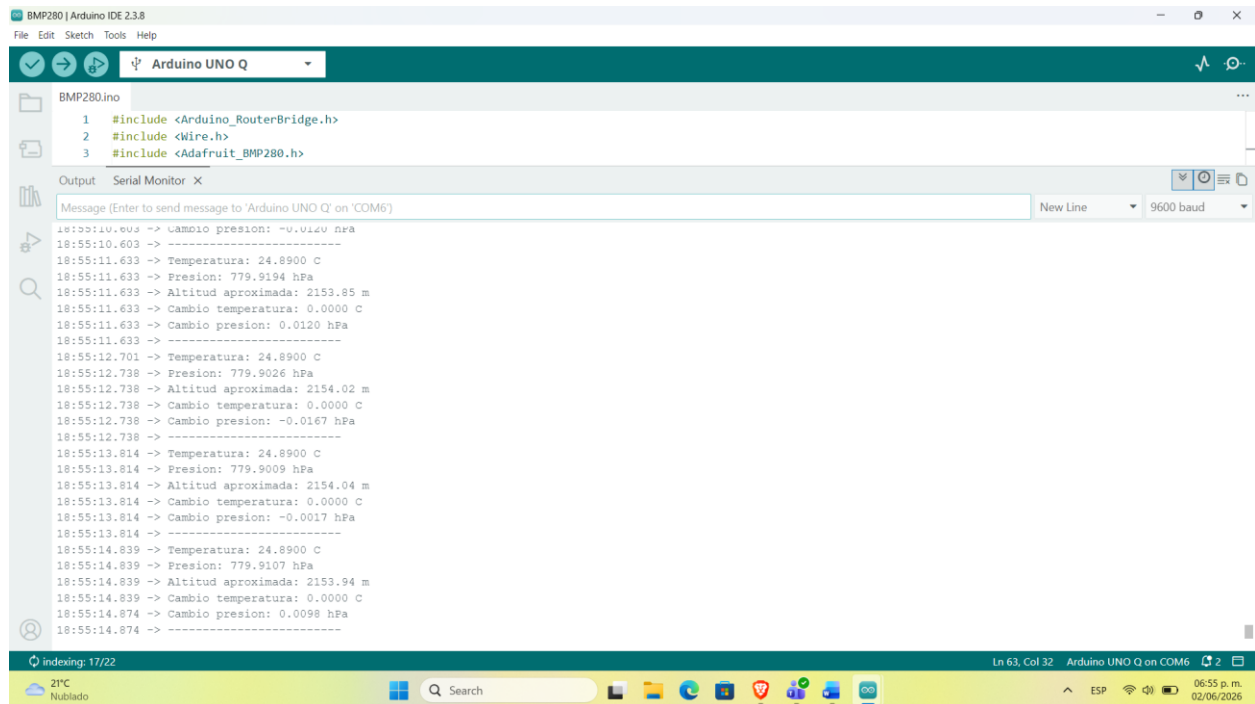
Output: Serial Monitor

```
Info : [stm32u5.ap0] gdb port disabled
Info : [stm32u5.cpu] starting gdb server on 3333
Info : Listening on port 3333 for gdb connections
CPU in Non-Secure state
[stm32u5.cpu] halted due to debug-request, current mode: Thread
xPSR: 0x41000000 pc: 0x08014f0a psp: 0x20021998
Info : device idcode = 0x30076482 (STM32U57/U58xx - Rev U : 0x3007)
Info : TZEN = 0 : TrustZone disabled by option bytes
Info : RDP level 0 (0xAA)
Info : Flash size = 2048 KiB
Info : Flash mode : dual-bank
Info : Padding image section 0 at 0x0811345c with 4 bytes (bank write end alignment)
Warn : Adding extra erase range, 0x08113460 .. 0x08113fff
```

Uploading... [CANCEL]

Código compilando y subiendo

Una vez que el Código se compilo y se cargo en la placa, podemos ver en la consola el código que se esta ejecutando, se observan las medidas que esta registrando



```
BMP280.ino
1 #include <Arduino_RouterBridge.h>
2 #include <Wire.h>
3 #include <Adafruit_BMP280.h>

Output Serial Monitor X
Message (Enter to send message to 'Arduino UNO Q' on 'COM6')
New Line 9600 baud

18:55:10.603 -> Cambio presion: -0.0120 hPa
18:55:11.633 -> Temperatura: 24.8900 C
18:55:11.633 -> Presion: 779.9194 hPa
18:55:11.633 -> Altitud aproximada: 2153.85 m
18:55:11.633 -> Cambio temperatura: 0.0000 C
18:55:11.633 -> Cambio presion: 0.0120 hPa
18:55:11.633 -> -----
18:55:12.701 -> Temperatura: 24.8900 C
18:55:12.738 -> Presion: 779.9026 hPa
18:55:12.738 -> Altitud aproximada: 2154.02 m
18:55:12.738 -> Cambio temperatura: 0.0000 C
18:55:12.738 -> Cambio presion: -0.0167 hPa
18:55:12.738 -> -----
18:55:13.814 -> Temperatura: 24.8900 C
18:55:13.814 -> Presion: 779.9009 hPa
18:55:13.814 -> Altitud aproximada: 2154.04 m
18:55:13.814 -> Cambio temperatura: 0.0000 C
18:55:13.814 -> Cambio presion: -0.0017 hPa
18:55:13.814 -> -----
18:55:14.839 -> Temperatura: 24.8900 C
18:55:14.839 -> Presion: 779.9107 hPa
18:55:14.839 -> Altitud aproximada: 2153.94 m
18:55:14.839 -> Cambio temperatura: 0.0000 C
18:55:14.874 -> Cambio presion: 0.0098 hPa
18:55:14.874 -> -----
```

Código ejecutando

Prueba unitaria sensor DS18B20

Para este sensor, es mas sencillo, ya que capturaremos la salida del sensor, es decir, se hará uso de los pines “DIGITAL IN” de nuestra Arduino Uno Q ya que se lee un valor digital desde el sensor, esta configuración se observa continuación el siguiente diagrama.

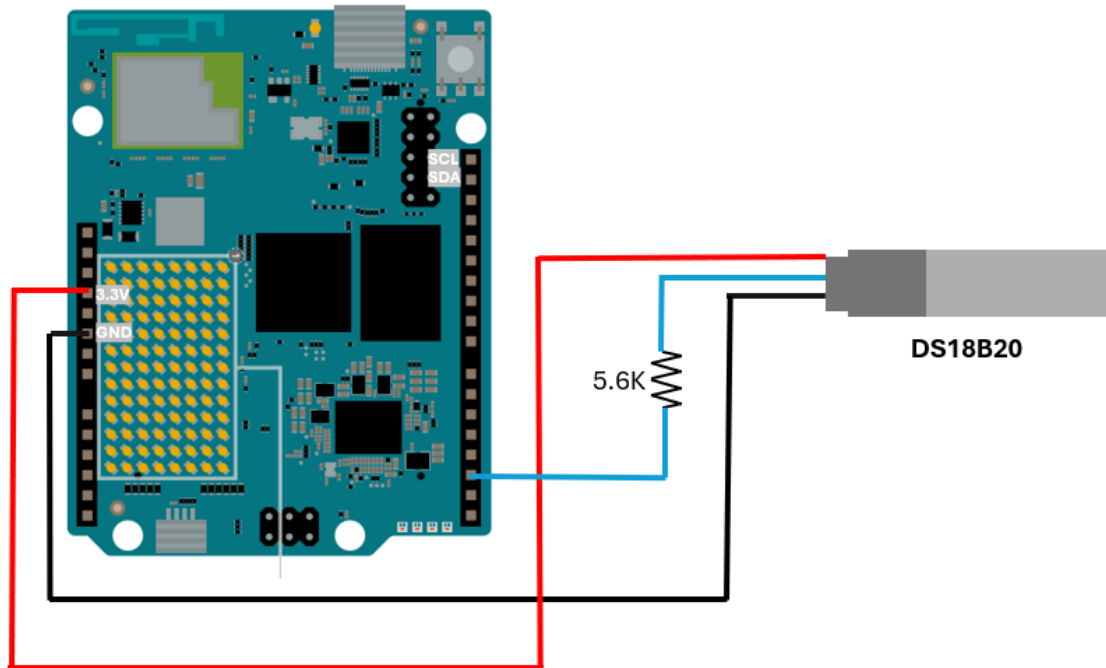
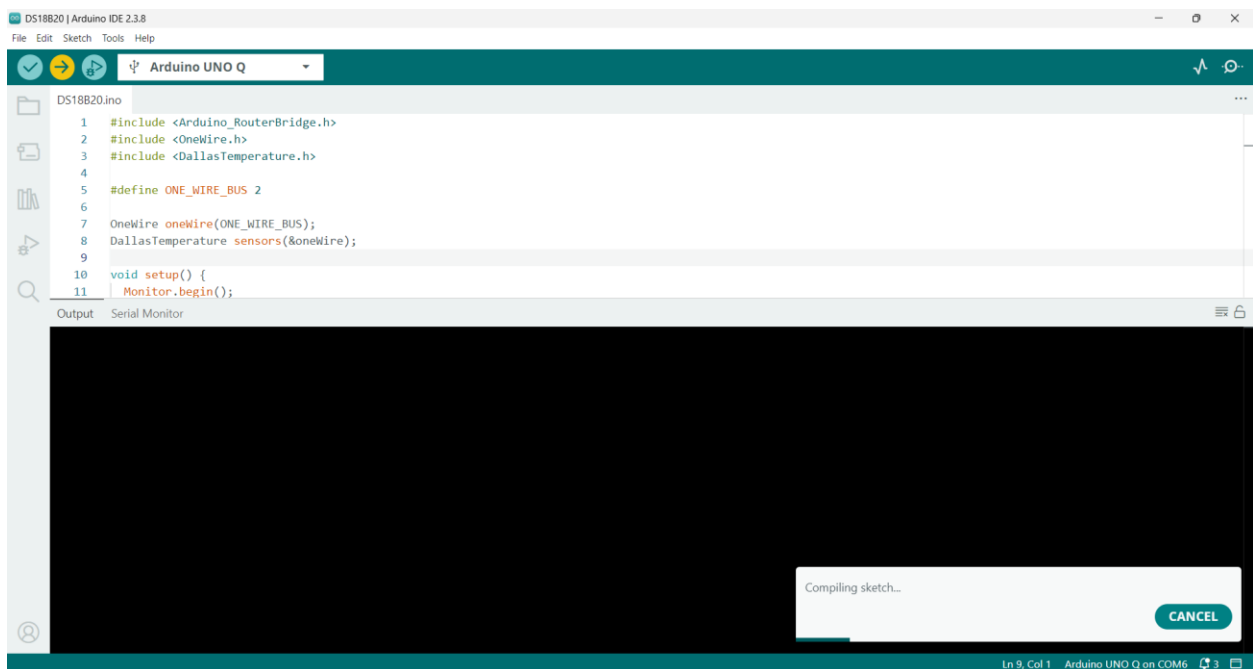
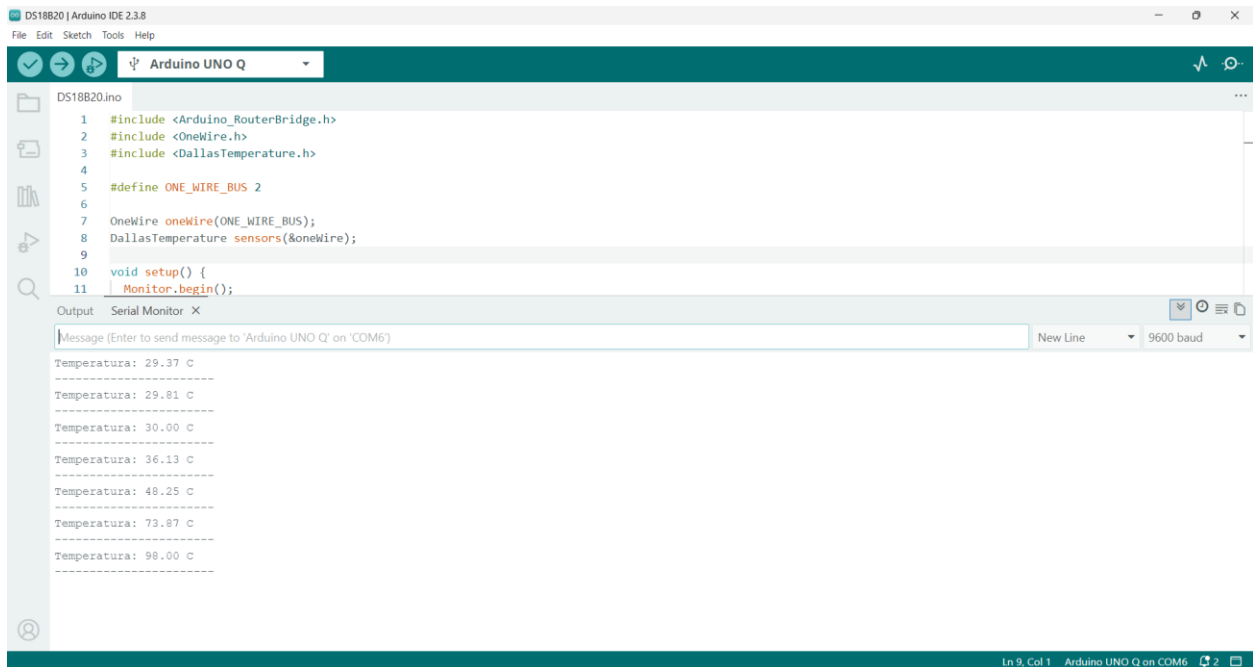


Diagrama de conexión Arduino Uno Q y sensor DS18B20

Una vez que se tienen las conexiones, podemos compilar y cargar el Código de la prueba del sensor en el IDE de desarrollo de Arduino, para consultar el Código se puede acceder en el repositorio.



Ahora que ya se cargó el código en nuestra placa, podemos pasar a ejecutar el código y probar nuestro sensor, como se podrá observar se muestra una temperatura de 29 grados, pero al acercar un encendedor al sensor, se puede observar como es que la temperatura va aumentando gradualmente con el tiempo.



The screenshot shows the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar has icons for checking, running, and uploading code. The board selected is 'Arduino UNO Q'. The code editor displays the following code for DS18B20.ino:

```
1 #include <Arduino_RouterBridge.h>
2 #include <OneWire.h>
3 #include <DallasTemperature.h>
4
5 #define ONE_WIRE_BUS 2
6
7 OneWire oneWire(ONE_WIRE_BUS);
8 DallasTemperature sensors(&oneWire);
9
10 void setup() {
11   Monitor.begin();
```

The Serial Monitor is open, showing the output of the program. The output displays the temperature in degrees Celsius, which increases from 29.37 to 98.00 as a heat source is applied.

Temperature (C)
29.37
29.81
30.00
36.13
48.25
73.87
98.00

Demostración de funcionamiento del sensor DS18B20

Prueba unitaria sensor MC-38

Para este sensor, se le piensa simular una puerta, una compuerta o una parte del contenedor de la cadena fría, para poder identificar rápidamente si el contenedor se abrió y desde ahí ocurrió el problema o es debido a otra situación, el diagrama de conexión de este

sensor es el siguiente.

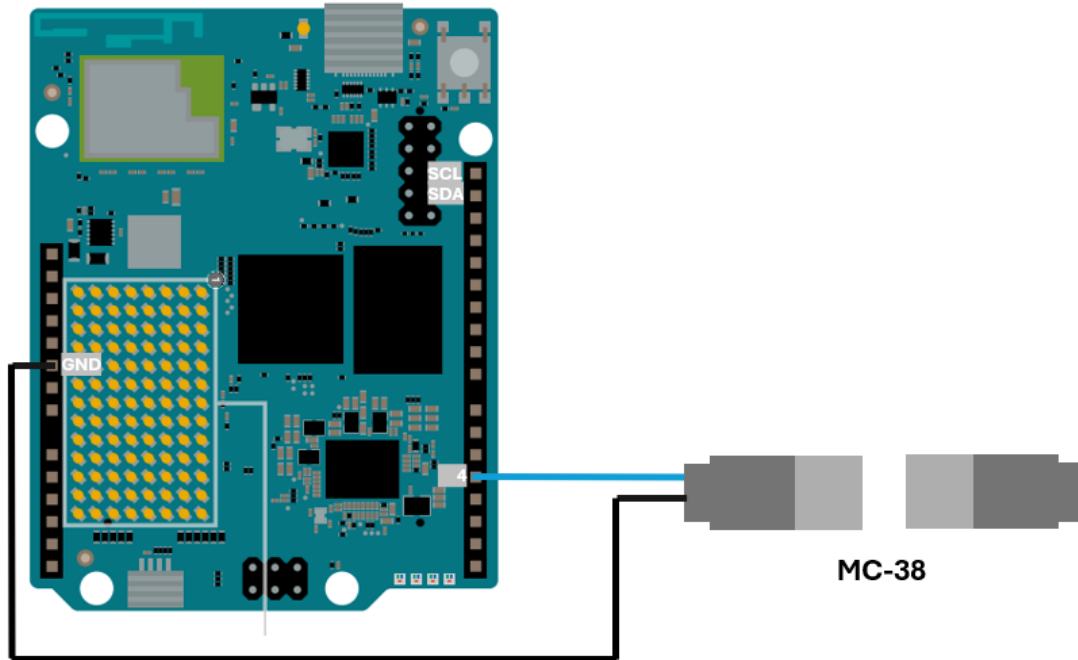


Diagrama de conexión Arduino Uno Q y sensor MC-38

Ahora hacemos la ejecución del programa después de compilar y cargarlo en nuestra placa, para este punto lo podemos observar en la siguiente imagen, como es que muestra si la puerta está abierta o cerrada.

La imagen muestra la interfaz de usuario de Arduino IDE 2.3.8. En la parte superior, se ve el archivo "MC-38.ino" cargado. El código fuente es el siguiente:

```
10 pinMode(PIN_MC38, INPUT_PULLUP);
11 }
12
13 void loop() {
14   int estado = digitalRead(PIN_MC38);
15
16   Serial.print("Lectura digital: ");
```

Debajo del código, se encuentra el "Serial Monitor" configurado para "Arduino UNO Q" en "COM6" con una velocidad de 9600 baud. El monitor muestra una serie de lecturas digitales que alternan entre 1 (ABIERTO) y 0 (CERRADO), indicando el estado de la puerta.

```
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 0 -> CERRADO - La puerta fue cerrada
Lectura digital: 0 -> CERRADO - La puerta fue cerrada
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 0 -> CERRADO - La puerta fue cerrada
Lectura digital: 0 -> CERRADO - La puerta fue cerrada
Lectura digital: 0 -> CERRADO - La puerta fue cerrada
Lectura digital: 0 -> CERRADO - La puerta fue cerrada
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
Lectura digital: 1 -> ABIERTO - La puerta fue abierta
```

Integración de sensores y comunicación MCU-MPU

(Arduino Uno Q)

La placa Arduino UNO Q integra en una sola tarjeta dos componentes principales: un microprocesador (MPU) y un microcontrolador (MCU). Esta arquitectura permite combinar tareas de alto nivel, como conectividad, procesamiento y ejecución de aplicaciones, con tareas de bajo nivel orientadas a la lectura de sensores y control de periféricos.

El MPU de la Arduino UNO Q está basado en un procesador Qualcomm Dragonwing QRB2210, con arquitectura Arm Cortex-A53 de cuatro núcleos, capaz de ejecutar un sistema operativo Linux basado en Debian. Este componente puede encargarse de tareas más complejas, como conexión a red, ejecución de aplicaciones, procesamiento de datos, almacenamiento y comunicación con servicios externos.

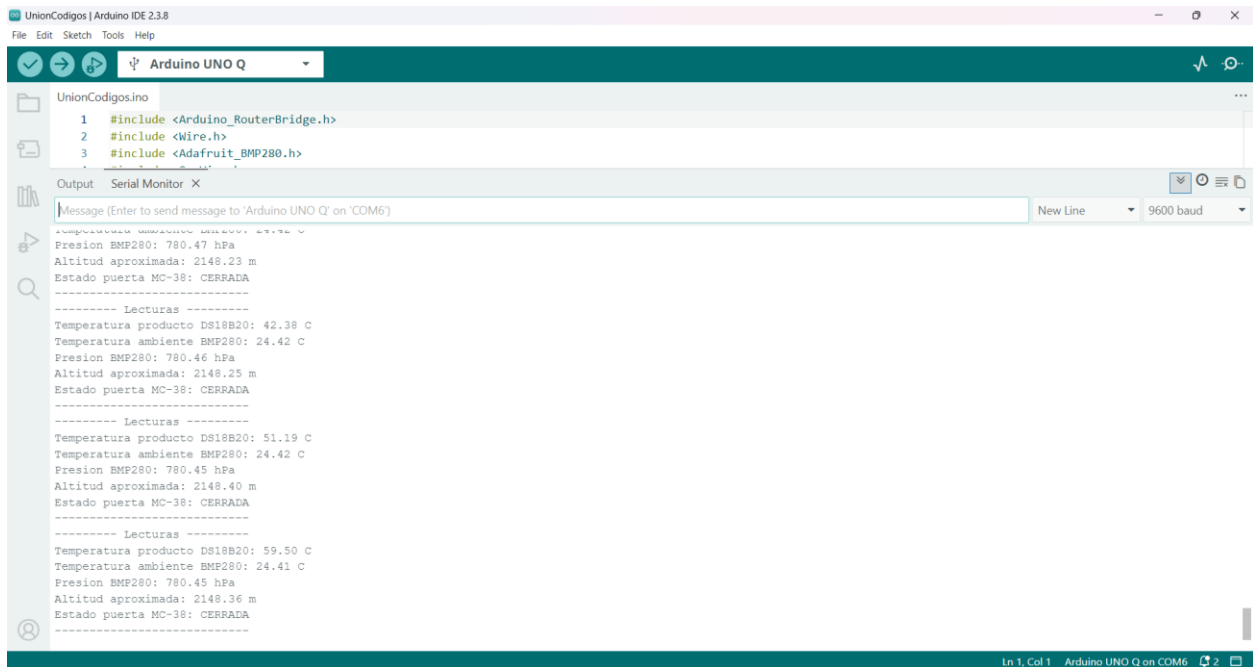
Por otro lado, el MCU corresponde a un microcontrolador STMicroelectronics STM32U585, basado en Arm Cortex-M33, encargado de ejecutar sketches de Arduino y realizar operaciones cercanas al hardware. En el contexto del proyecto, el MCU se utiliza para leer sensores como el DS18B20, BMP280, MC-38 y MPU6050, debido a que estos requieren interacción directa con pines digitales, buses de comunicación y señales físicas.

La comunicación entre el MPU y el MCU se realiza mediante Bridge, una librería de Arduino basada en RPC, es decir, llamadas a procedimientos remotos. Esto permite que los programas ejecutados en el microcontrolador puedan acceder a servicios del sistema Linux, mientras que las aplicaciones del sistema Linux pueden interactuar con los periféricos controlados por el microcontrolador.

En este proyecto, esta arquitectura resulta útil porque permite separar responsabilidades. El MCU se encarga de la adquisición de datos en tiempo real desde los sensores, mientras que el MPU puede encargarse de tareas de comunicación, publicación MQTT, almacenamiento o integración con aplicaciones de visualización. De esta forma, la Arduino UNO Q funciona como un nodo IoT completo, capaz de medir variables físicas y transmitir las hacia un broker MQTT para su posterior análisis o monitoreo.

Lectura de los 3 sensores

Así que ahora juntamos el código de los sensores, para poder tener únicamente 1 único código, el cual es el código que se cargara en nuestra placa Arduino Uno Q, el cual después de compilarlo y cargarlo a nuestra placa nos muestra en consola lo siguiente.



```
UnionCódigos.ino
1 #include <Arduino_RouterBridge.h>
2 #include <Wire.h>
3 #include <Adafruit_BMP280.h>

Output Serial Monitor X
Message (Enter to send message to 'Arduino UNO Q' on 'COM6')
New Line 9600 baud

Presion BMP280: 780.47 hPa
Altitud aproximada: 2148.23 m
Estado puerta MC-38: CERRADA
----- Lecturas -----
Temperatura producto DS18B20: 42.38 C
Temperatura ambiente BMP280: 24.42 C
Presion BMP280: 780.46 hPa
Altitud aproximada: 2148.25 m
Estado puerta MC-38: CERRADA
----- Lecturas -----
Temperatura producto DS18B20: 51.19 C
Temperatura ambiente BMP280: 24.42 C
Presion BMP280: 780.45 hPa
Altitud aproximada: 2148.40 m
Estado puerta MC-38: CERRADA
----- Lecturas -----
Temperatura producto DS18B20: 59.50 C
Temperatura ambiente BMP280: 24.41 C
Presion BMP280: 780.45 hPa
Altitud aproximada: 2148.36 m
Estado puerta MC-38: CERRADA
```

Resultado de lecturas de los 3 sensores utilizados.

Bridge / RPC

El Bridge/RPC funciona como una comunicación interna entre el programa del MCU y el programa del MPU. Arduino lo describe como una librería que permite que aplicaciones en Linux llamen funciones del microcontrolador, y que sketches del microcontrolador puedan acceder a servicios del sistema Linux. Es importante porque el MPU no lee directamente los pines del Arduino. Los headers de Arduino están ligados al MCU, así que lo correcto es que el MCU lea los sensores y el MPU consuma esos datos por Bridge/RPC. Esta separación coincide con la arquitectura de la UNO Q: sensores y control en MCU; Linux, conectividad y aplicaciones en MPU.

Contrato de comunicación MCU-MPU

Para la comunicación interna entre el microcontrolador y el microprocesador de la Arduino UNO Q se definió un contrato basado en una arquitectura por capas. El MCU será responsable de la adquisición de datos desde los sensores físicos, mientras que el MPU será

responsable de construir el mensaje final, agregar información de sistema y publicar los datos mediante MQTT.

Responsabilidades del MCU

El MCU se encarga de leer directamente los sensores conectados a la Arduino UNO Q:

- DS18B20: temperatura del producto o contenedor.
- BMP280: temperatura ambiente, presión atmosférica y altitud aproximada.
- MC-38: estado de apertura o cierre de la puerta/tapa del contenedor.

El MCU no publica mensajes MQTT. Su función principal es obtener las lecturas, validar si los sensores están disponibles y exponer la última medición al MPU mediante Bridge/RPC.

Responsabilidades del MPU

El MPU ejecuta la lógica de comunicación de red. Su función es solicitar al MCU la última lectura disponible, agregar un timestamp del sistema Linux, construir un mensaje JSON y publicarlo en el broker MQTT.

El MPU también puede encargarse de evaluar reglas de negocio, por ejemplo, determinar si existe una alerta por temperatura alta o puerta abierta.

Modelo de comunicación

La comunicación se realizará bajo un modelo de solicitud-respuesta. El MPU solicitará periódicamente la lectura actual al MCU, y el MCU responderá con el último paquete de datos disponible.

El MCU no debe bloquearse esperando a que el MPU solicite información. En su lugar, el MCU debe mantener una lectura actualizada en memoria. Cuando el MPU invoque la función de lectura, el MCU devolverá la última muestra registrada.

Servicio RPC propuesto

Nombre del servicio: **coldchain.mcu.v1**

Método principal: **get_reading**

Descripción, permite que el MPU solicite al MCU la última lectura registrada de los sensores conectados al sistema de cadena de frío.

La solicitud del MPU al MCU

La solicitud desde el microprocesador hacia el microcontrolador se plantea que sea de la siguiente manea.



```
{
  "method": "coldchain.mcu.v1.get_reading",
  "params": {
    "request_id": "req-001"
  }
}
```

formato petición del MPU al MCU

Respuesta del MCU al MPU

Ahora se plantea el formato de la respuesta hacia el microprocesador, cuando el microcontrolador tenga que responder con los valores capturados de los sensores.



```
{
  "schema_version": "1.0",
  "request_id": "req-001",
  "sample_id": 152,
  "mcu_uptime_ms": 304500,
  "data": {
    "temperatura_producto_c": 4.8,
    "temperatura_ambiente_c": 26.7,
    "presion_hpa": 784.95,
    "altitud_m": 2102.11,
    "puerta": "cerrada",
    "puerta_raw": 0
  },
  "valid": {
    "ds18b20": true,
    "bmp280": true,
    "mc38": true
  },
  "errors": []
}
```

Formato respuesta del MCU al MPU.

En donde el significado de cada uno de los campos seria el siguiente

Campo	Tipo	Responsable	Descripción
schema_version	texto	MCU	Versión del contrato de datos
request_id	texto	MPU/MCU	Identificador de la solicitud
sample_id	entero	MCU	Número consecutivo de muestra
mcu_uptime_ms	entero	MCU	Tiempo activo del MCU en milisegundos
temperatura_producto_c	decimal	MCU	Temperatura medida por el DS18B20
temperatura_ambiente_c	decimal	MCU	Temperatura medida por el BMP280
presion_hpa	decimal	MCU	Presión atmosférica medida por el BMP280
altitud_m	decimal	MCU	Altitud aproximada calculada con el BMP280
puerta	texto	MCU	Estado lógico del MC-38: abierta o cerrada
puerta_raw	entero	MCU	Lectura digital directa del pin
valid	objeto	MCU	Indica si cada sensor entregó datos válidos
errors	arreglo	MCU	Lista de errores detectados durante la lectura

Manejo de errores

Si un sensor falla, el MCU debe responder sin detener todo el sistema. El valor del sensor fallido se enviará como null, su bandera dentro de valid será false, y se agregará un código de error en el arreglo errors. Ejemplo de error por DS18B20 desconectado

```
{
  "schema_version": "1.0",
  "request_id": "req-002",
  "sample_id": 153,
  "mcu_uptime_ms": 306500,
  "data": {
    "temperatura_producto_c": null,
    "temperatura_ambiente_c": 26.8,
    "presion_hpa": 784.90,
    "altitud_m": 2102.45,
    "puerta": "cerrada",
    "puerta_raw": 0
  },
  "valid": {
    "ds18b20": false,
    "bmp280": true,
    "mc38": true
  },
  "errors": [
    {
      "sensor": "ds18b20",
      "code": "DS18B20_DISCONNECTED",
      "message": "No se pudo obtener lectura valida del sensor DS18B20"
    }
  ]
}
```

Ejemplo de respuesta de error por el sensor DS18B20

Datos agregados por el MPU

El MPU no modifica las lecturas originales del MCU. Sin embargo, antes de publicar por MQTT, agrega datos propios del sistema Linux y de la comunicación MQTT. Ejemplo del mensaje final publicado por el MPU

```
{
  "schema_version": "1.0",
  "device_id": "unoq-cadena-frio-01",
  "timestamp": "2026-06-21T03:15:00Z",
  "source":
  {
    "board": "Arduino UNO Q",
    "mcu": "STM32U585",
    "mpu": "Qualcomm QRB2210"
  },
  "data":
  {
    "temperatura_producto_c": 4.8,
    "temperatura_ambiente_c": 26.7,
    "presion_hpa": 784.95,
    "altitud_m": 2102.11,
    "puerta": "cerrada"
  },
  "estado": "NORMAL"
}
```

Ejemplo de datos agregados por el MPU

Reglas de estado del sistema

El MPU puede determinar el estado general del sistema a partir de las lecturas recibidas.

Condición	Estado
Temperatura dentro del rango y puerta cerrada	NORMAL
Temperatura del producto mayor al límite permitido	ALERTA_TEMPERATURA
Puerta abierta	ALERTA_PUERTA_ABIERTA
Temperatura fuera de rango y puerta abierta	ALERTA_GENERAL
Algún sensor crítico sin lectura válida	ERROR_SENSOR

Frecuencia de lectura

El MCU actualizará las lecturas de sensores cada 2 segundos. El MPU solicitará la última lectura disponible cada 2 segundos y publicará el mensaje MQTT correspondiente.

Para evitar bloqueos, la función RPC del MCU debe devolver la última lectura almacenada, no iniciar una lectura completa desde cero en cada solicitud del MPU.

Tópicos MQTT asociados

Aunque el contrato MCU-MPU no publica directamente en MQTT, los datos recibidos por el MPU serán usados para publicar en los siguientes tópicos.

Tópico	Uso
escom/iot/equipoX/cadena_frio/sensores	Publicación periódica de lecturas
escom/iot/equipoX/cadena_frio/estado	Estado general del sistema
escom/iot/equipoX/cadena_frio/alertas	Alertas por temperatura, puerta abierta o error de sensor

Diagrama general de comunicación entre MCU y MPU

Finalmente, la comunicación queda de la siguiente manera, en el cual se cuenta con los sensores, la placa de desarrollo Arduino Uno Q y MQTT.

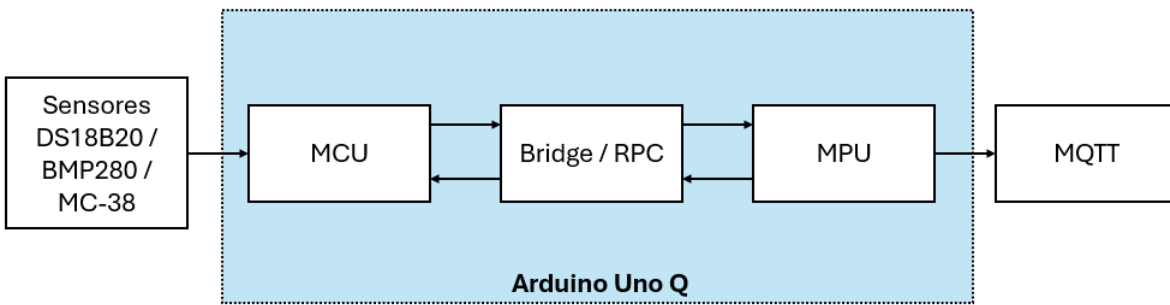


Diagrama de comunicación MCU con MPU

Programa MCU con cache

Para este punto ya tenemos nuestro código unido, el cual lee los valores de los 3 sensores, sin embargo, como se definió anteriormente, ya se tiene el contrato de comunicación entre el MCU y el MPU, por lo cual ahora se reestructura el código para que cumpla con lo que se definió, este código hará las siguientes cosas

- Leerá los sensores cada cierto tiempo.
- Guardará la última lectura en una estructura cacheada
- Expondrá esa última lectura al MPU por Bridge/RPC, sin que el MPU tenga que leer sensores directamente

El código se podrá consultar en el repositorio, por lo que ahora después de tener el código en nuestro IDE vamos a compilar y cargar en nuestra placa de desarrollo, y como hasta este punto no tenemos el código en el MPU, observaremos el resultado en la consola, como se muestra a continuación.

The screenshot shows the Arduino IDE 2.3.8 interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar shows icons for opening files, saving, and running the sketch. The file explorer on the left shows the project files, including MCU_cache.ino. The main editor window displays the code for MCU_cache.ino, which includes the following headers:

```
1 #include <Arduino_RouterBridge.h>
2 #include <Wire.h>
3 #include <Adafruit_BMP280.h>
4 #include <OneWire.h>
5 #include <DallasTemperature.h>
```

The Serial Monitor is open, showing the output of the sketch. The output includes sensor readings and JSON data. The JSON data is as follows:

```
JSON cacheado: {"schema_version":"1.0","sample_id":69,"mcu_uptime_ms":137433,"last_update_ms":137433,"data":{"temperatura_producto_c":21.19,"temperatura_ambiente_c":24.27,
-----
CACHE MCU -----
sample_id: 70
DS18B20 temp producto: 21.19 C
BMP280 temp ambiente: 24.28 C
BMP280 presion: 781.17 hPa
BMP280 altitud: 2140.95 m
MC-38 puerta: cerrada | raw: 0
JSON cacheado: {"schema_version":"1.0","sample_id":70,"mcu_uptime_ms":139433,"last_update_ms":139433,"data":{"temperatura_producto_c":21.19,"temperatura_ambiente_c":24.28,
-----
CACHE MCU -----
sample_id: 71
DS18B20 temp producto: 21.19 C
BMP280 temp ambiente: 24.28 C
BMP280 presion: 781.18 hPa
BMP280 altitud: 2140.85 m
MC-38 puerta: cerrada | raw: 0
JSON cacheado: {"schema_version":"1.0","sample_id":71,"mcu_uptime_ms":141433,"last_update_ms":141433,"data":{"temperatura_producto_c":21.19,"temperatura_ambiente_c":24.28,
```

Resultado de ejecución del código MCU_cache

Se observa que el JSON generado es el siguiente.


```
{
  "schema_version":"1.0",
  "sample_id":110,
  "mcu_uptime_ms":219440,
  "last_update_ms":219440,
  "data":{
    "temperatura_producto_c":21.19,
    "temperatura_ambiente_c":24.29,
    "presion_hpa":781.18,
    "altitud_m":2140.86,
    "puerta":"cerrada",
    "puerta_raw":0
  },
  "valid":{
    "ds18b20":true,
    "bmp280":true,
    "mc38":true
  },
  "errors":[
  ]
}
```

Resultado JSON de ejecución del código MCU_cache

Ahora se realizará la desconexión del sensor DS18B20, y observaremos cual es el error que manda desde la consola.

```
MCU_cache.ino
1 #include <Arduino_RouterBridge.h>
2 #include <Wire.h>
3 #include <Adafruit_BMP280.h>
4 #include <OneWire.h>
5 #include <DallasTemperature.h>
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino UNO Q' on 'COM6')

New Line 9600 baud

BMP280 presion: 781.23 hPa
BMP280 altitud: 2140.42 m
MC-38 puerta: cerrada | raw: 0
JSON cacheado: {"schema_version":"1.0","sample_id":176,"mcu_uptime_ms":351447,"last_update_ms":351447,"data":{"temperatura_producto_c":null,"temperatura_ambiente_c":24.34,

CACHE MCU -----
sample_id: 177
DS18B20 temp producto: ERROR
BMP280 temp ambiente: 24.34 C
BMP280 presion: 781.19 hPa
BMP280 altitud: 2140.80 m
MC-38 puerta: cerrada | raw: 0
JSON cacheado: {"schema_version":"1.0","sample_id":177,"mcu_uptime_ms":353447,"last_update_ms":353447,"data":{"temperatura_producto_c":null,"temperatura_ambiente_c":24.34,

CACHE MCU -----
sample_id: 178
DS18B20 temp producto: ERROR
BMP280 temp ambiente: 24.34 C
BMP280 presion: 781.20 hPa
BMP280 altitud: 2140.66 m
MC-38 puerta: cerrada | raw: 0
JSON cacheado: {"schema_version":"1.0","sample_id":178,"mcu_uptime_ms":355447,"last_update_ms":355447,"data":{"temperatura_producto_c":null,"temperatura_ambiente_c":24.34,

Ln 1, Col 1 Arduino UNO Q on COM6

Resultado error en sensor de ejecución del código MCU_cache

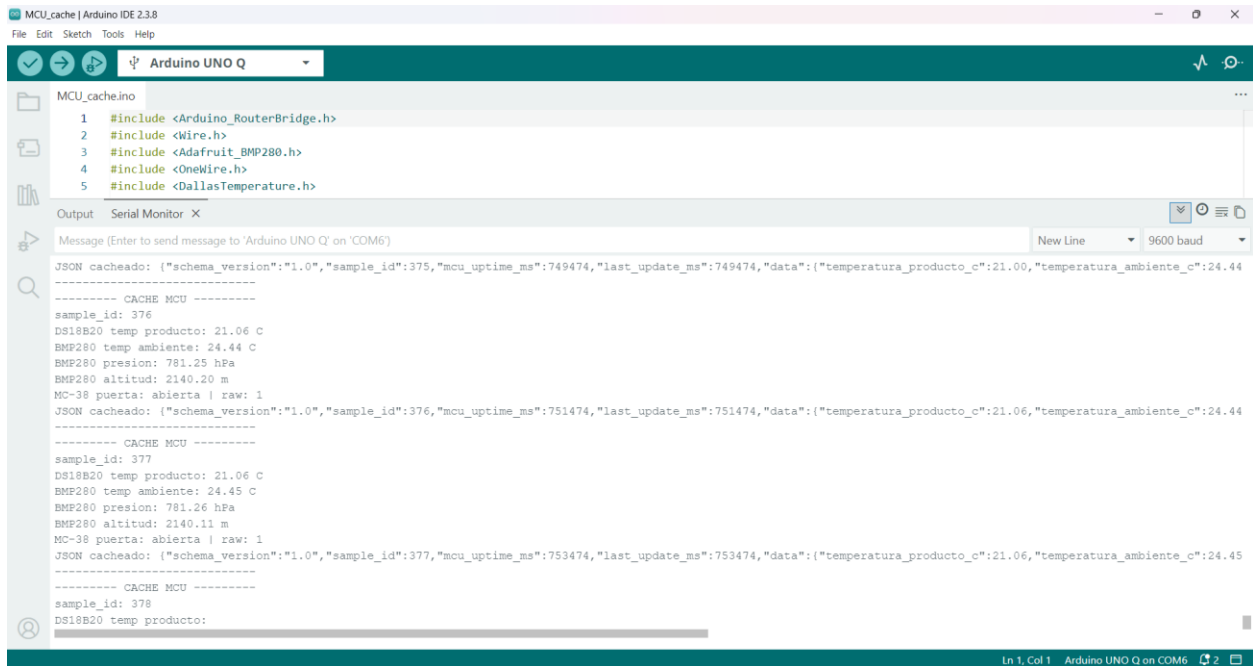
Y ahora se muestra el mensaje que genera de error cuando un sensor está dañado o no entrega una lectura.

```
{
  "schema_version": "1.0",
  "sample_id": 195,
  "mcu_uptime_ms": 389449,
  "last_update_ms": 389449,
  "data": {
    "temperatura_producto_c": null,
    "temperatura_ambiente_c": 24.35,
    "presion_hpa": 781.20,
    "altitud_m": 2140.64,
    "puerta": "cerrada",
    "puerta_raw": 0
  },
  "valid": {
    "ds18b20": false,
    "bmp280": true,
    "mc38": true
  },
  "errors": [
    {
      "sensor": "ds18b20",
      "code": "DS18B20_DISCONNECTED",
      "message": "No se pudo obtener lectura valida del DS18B20"
    }
  ]
}
```

Resultado JSON de un error en sensor en la ejecución del código MCU_cache

Además, también se puede observar el cambio en el valor de las variables cuando el sensor de la puerta marca que está en otro estado, en este caso vamos a probar en

abierto.



The screenshot shows the Arduino IDE interface. The top toolbar includes icons for checking, uploading, and running the code, along with a dropdown menu for the board, currently set to 'Arduino UNO Q'. The left sidebar shows a file explorer with 'MCU_cache.ino' selected. The main editor window displays the code for 'MCU_cache.ino', which includes several library headers: `#include <Arduino_RouterBridge.h>`, `#include <Wire.h>`, `#include <Adafruit_BMP280.h>`, `#include <OneWire.h>`, and `#include <DallasTemperature.h>`. The bottom panel shows the 'Serial Monitor' window, which displays the output of the code. The output includes a JSON message sent to 'Arduino UNO Q' on 'COM6', followed by a series of sensor readings and status updates. The status 'MC-38 puerta: abierta | raw: 1' is repeated multiple times, indicating the door is open. The JSON messages contain data such as 'temperatura_producto_c': 21.00, 'temperatura_ambiente_c': 24.44, and 'sample_id': 375, 376, 377, 378. The status bar at the bottom indicates 'Ln 1, Col 1' and 'Arduino UNO Q on COM6'.

```
MCU_cache.ino
1 #include <Arduino_RouterBridge.h>
2 #include <Wire.h>
3 #include <Adafruit_BMP280.h>
4 #include <OneWire.h>
5 #include <DallasTemperature.h>

Output Serial Monitor X
Message (Enter to send message to 'Arduino UNO Q' on 'COM6')
New Line 9600 baud

JSON cacheado: {"schema_version":"1.0","sample_id":375,"mcu_uptime_ms":749474,"last_update_ms":749474,"data":{"temperatura_producto_c":21.00,"temperatura_ambiente_c":24.44}}
----- CACHE MCU -----
sample_id: 376
DS18B20 temp producto: 21.06 C
BMF280 temp ambiente: 24.44 C
BMF280 presion: 781.25 hPa
BMF280 altitud: 2140.20 m
MC-38 puerta: abierta | raw: 1
JSON cacheado: {"schema_version":"1.0","sample_id":376,"mcu_uptime_ms":751474,"last_update_ms":751474,"data":{"temperatura_producto_c":21.06,"temperatura_ambiente_c":24.44}}
----- CACHE MCU -----
sample_id: 377
DS18B20 temp producto: 21.06 C
BMF280 temp ambiente: 24.45 C
BMF280 presion: 781.26 hPa
BMF280 altitud: 2140.11 m
MC-38 puerta: abierta | raw: 1
JSON cacheado: {"schema_version":"1.0","sample_id":377,"mcu_uptime_ms":753474,"last_update_ms":753474,"data":{"temperatura_producto_c":21.06,"temperatura_ambiente_c":24.45}}
----- CACHE MCU -----
sample_id: 378
DS18B20 temp producto:
```

Salida del codigo codigo MCU_cache

Y esto tenemos en el JSON, como resultado en puerta como “abierta”

```
{
  "schema_version": "1.0",
  "sample_id": 388,
  "mcu_uptime_ms": 775476,
  "last_update_ms": 775476,
  "data": {
    "temperatura_producto_c": 21.12,
    "temperatura_ambiente_c": 24.44,
    "presion_hpa": 781.27,
    "altitud_m": 2139.96,
    "puerta": "abierta",
    "puerta_raw": 1
  },
  "valid": {
    "ds18b20": true,
    "bmp280": true,
    "mc38": true
  },
  "errors": [
  ]
}
```

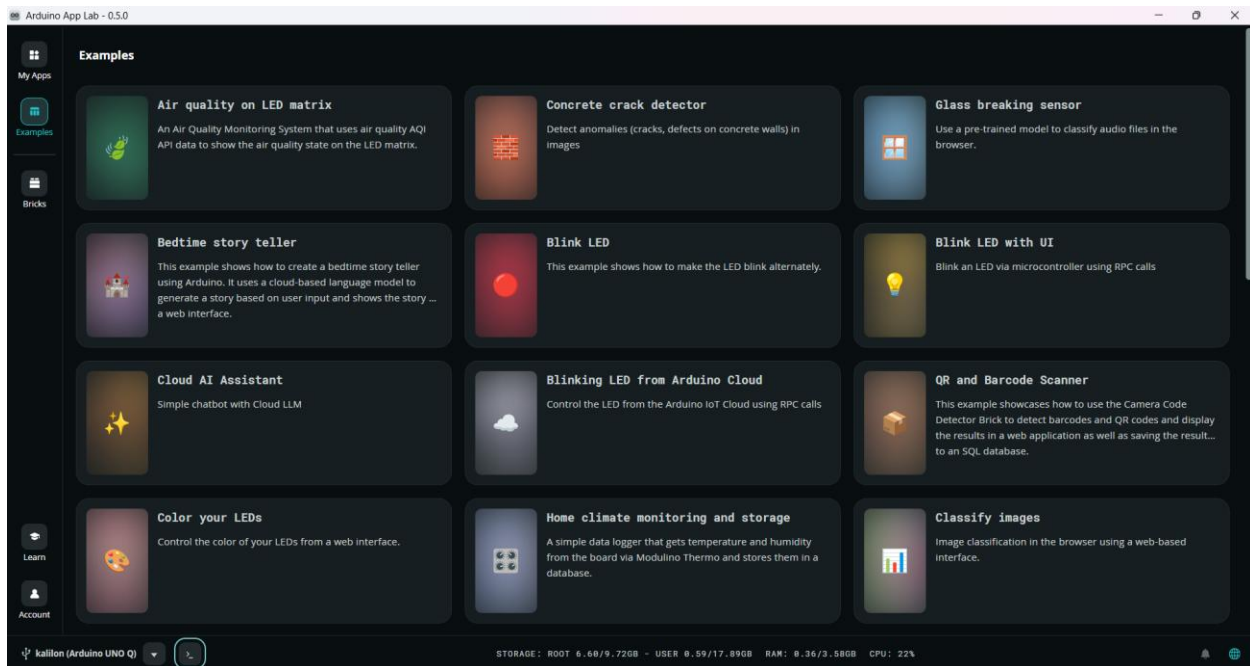
Salida del código en JSON del código MCU_cache

Programa MPU

Ahora vamos a crear el código para el MPU que será el encargado de pedir las lecturas al MCU y además de publicar por MQTT todos estos valores hacia Mosquitto.

En UNO Q, el lado Python corre en el MPU/Linux, y el sketch C++ corre en el MCU; la comunicación entre ambos se hace con `Bridge.call()` / `Bridge.provide()` mediante Arduino Router/Bridge. Además, Paho MQTT recomienda usar `connect()`, `loop_start()` y `publish()` para mantener la conexión MQTT y publicar mensajes.

Para esta parte del código es necesario tener instalado el IDE de Arduino App Lab, el cual nos permitirá conectarnos con el MPU de nuestra placa, una vez dentro del IDE, vamos a crear nuestra carpeta y crear el script que tendrá la comunicación con nuestro MCU.



Al abrir la aplicación, en la parte de hasta abajo nos aparecerá la opción de abrir la bash de nuestra placa, así que hacemos clic y creamos lo necesario.

```

arduino@kallion: ~/proyecto
arduino@kallion:~$ ls
ArduinoApps  lost+found  sisemb
arduino@kallion:~$ mkdir proyecto_sistemas_embebidos
arduino@kallion:~$ cd proyecto_sistemas_embebidos/
arduino@kallion:~/proyecto_sistemas_embebidos$ python/requirements.txt
bash: python/requirements.txt: No such file or directory
arduino@kallion:~/proyecto_sistemas_embebidos$ touch requirements.txt
arduino@kallion:~/proyecto_sistemas_embebidos$ echo paho-mqtt>=2.0.0 > requirements.txt
arduino@kallion:~/proyecto_sistemas_embebidos$ cat requirements.txt
paho-mqtt
arduino@kallion:~/proyecto_sistemas_embebidos$ echo "paho-mqtt>=2.0.0" > requirements.txt
arduino@kallion:~/proyecto_sistemas_embebidos$ cat requirements.txt
paho-mqtt>=2.0.0
arduino@kallion:~/proyecto_sistemas_embebidos$ touch main.py
arduino@kallion:~/proyecto_sistemas_embebidos$ ls
'=2.0.0'  main.py  requirements.txt
arduino@kallion:~/proyecto_sistemas_embebidos$ |

```

Instalación Mosquitto en la UNO Q

Entramos a la terminal Linux de la UNO Q, no al sketch de Arduino. Si tienes modo computadora, monitor y teclado, puedes hacerlo directo. Si estás conectado por red, puede ser por SSH, en este caso al abrir el IDE de Arduino App Labs, te abre una terminal que esta conectada por SSH, así que usamos esta opción, ejecutamos el siguiente comando.

```
arduino@kali:~$ sudo apt update
Hit:1 http://deb.debian.org/debian trixie-backports InRelease
Hit:2 http://deb.debian.org/debian trixie InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
Hit:4 http://deb.debian.org/debian-security trixie-security InRelease
Hit:5 https://apt-repo.arduino.cc stable InRelease
146 packages can be upgraded. Run 'apt list --upgradable' to see them.
arduino@kali:~$ sudo apt install -y mosquitto mosquitto-clients
Building dependency tree... 0%
```

Creamos el siguiente fichero en sudo nano /etc/mosquitto/conf.d/cadena-frio.conf

```
arduino@kali:~$ sudo nano /etc/mosquitto/conf.d/cadena-frio.conf
arduino@kali:~$ |
```

y ahora agregamos las configuraciones

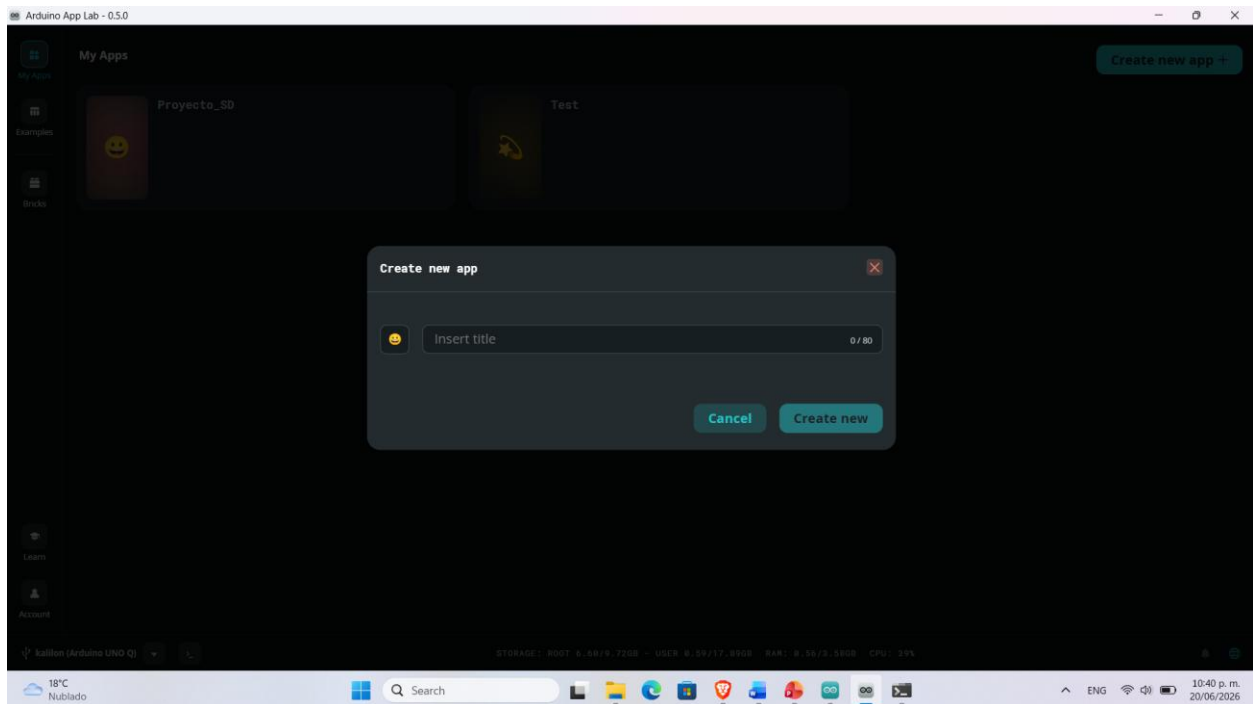
```
arduino@kallion: ~  
GNU nano 8.4 /etc/mosquitto/conf.d/cadena-frio.conf *  
listener 1883 0.0.0.0  
allow_anonymous true
```

En este paso es necesario reiniciar Mosquitto con el siguiente comando y comprobamos si está en ejecución.

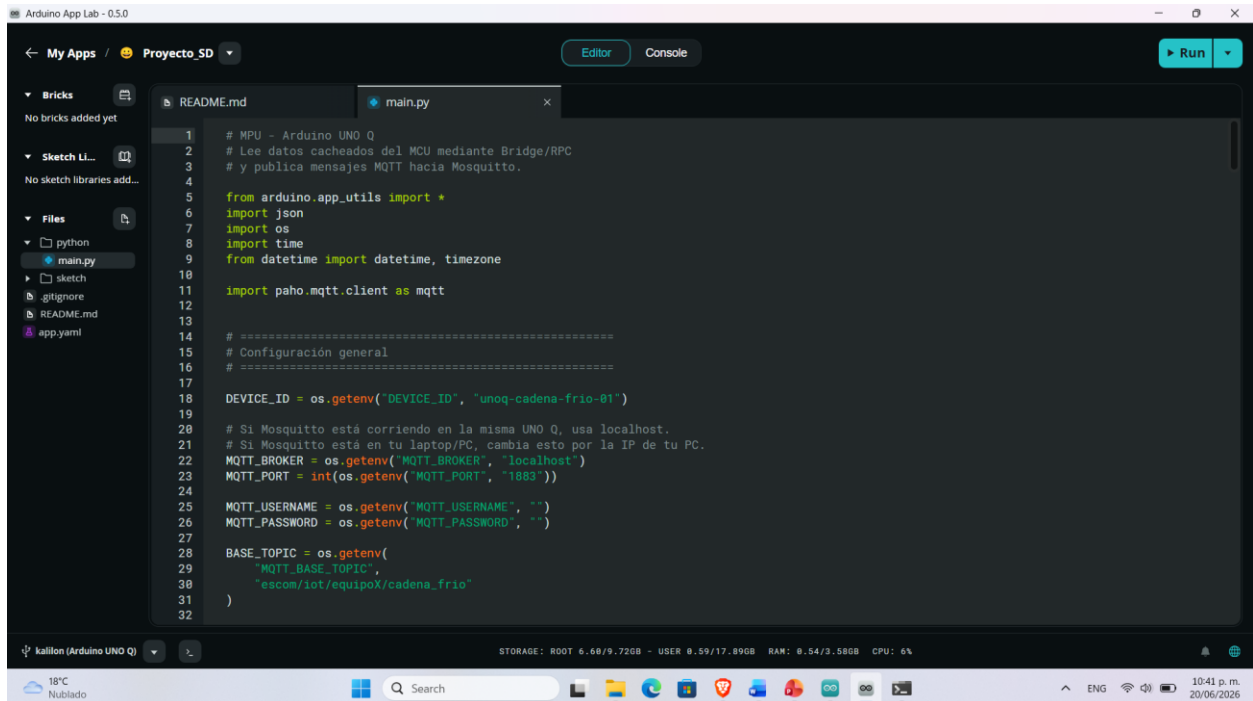
```
arduino@kallion: ~  
arduino@kallion:~$ sudo systemctl reset-failed mosquitto  
arduino@kallion:~$ sudo systemctl restart mosquitto  
arduino@kallion:~$ sudo systemctl status mosquitto --no-pager -l  
● mosquitto.service - Mosquitto MQTT Broker  
   Loaded: loaded (/usr/lib/systemd/system/mosquitto.service; enabled; preset: enabled)  
   Active: active (running) since Sun 2026-06-21 05:06:42 UTC; 3s ago  
     Invocation: b9b7a1e7bec04a6ca6203d2c111263e4  
       Docs: man:mosquitto.conf(5)  
            man:mosquitto(8)  
    Process: 16521 ExecStartPre=/bin/mkdir -m 740 -p /var/log/mosquitto (code=exited, status=0/SUCCESS)  
    Process: 16523 ExecStartPre=/bin/chown mosquitto:mosquitto /var/log/mosquitto (code=exited, status=0/SUCCESS)  
    Process: 16525 ExecStartPre=/bin/mkdir -m 740 -p /run/mosquitto (code=exited, status=0/SUCCESS)  
    Process: 16527 ExecStartPre=/bin/chown mosquitto:mosquitto /run/mosquitto (code=exited, status=0/SUCCESS)  
   Main PID: 16530 (mosquitto)  
      Tasks: 1 (limit: 4585)  
     Memory: 1.3M (peak: 1.8M)  
        CPU: 178ms  
    CGroup: /system.slice/mosquitto.service  
            └─16530 /usr/sbin/mosquitto -c /etc/mosquitto/mosquitto.conf  
  
Jun 21 05:06:42 kallion systemd[1]: Starting mosquitto.service - Mosquitto MQTT Broker...  
Jun 21 05:06:42 kallion mosquitto[16530]: 1782018402: Loading config file /etc/mosquitto/conf.d/default.conf  
Jun 21 05:06:42 kallion systemd[1]: Started mosquitto.service - Mosquitto MQTT Broker.  
arduino@kallion:~$ ss -ltnp | grep 1883  
LISTEN 0      100        0.0.0.0:1883      0.0.0.0:*  
LISTEN 0      100        [::]:1883        [::]:*  
arduino@kallion:~$ mosquitto_sub -h localhost -t "test/#" -v  
test/hola hola desde la UNO Q  
|
```

Ahora si vamos a pegar nuestro código que tenemos para nuestro MPU, el cual se puede consultar en el repositorio del código.

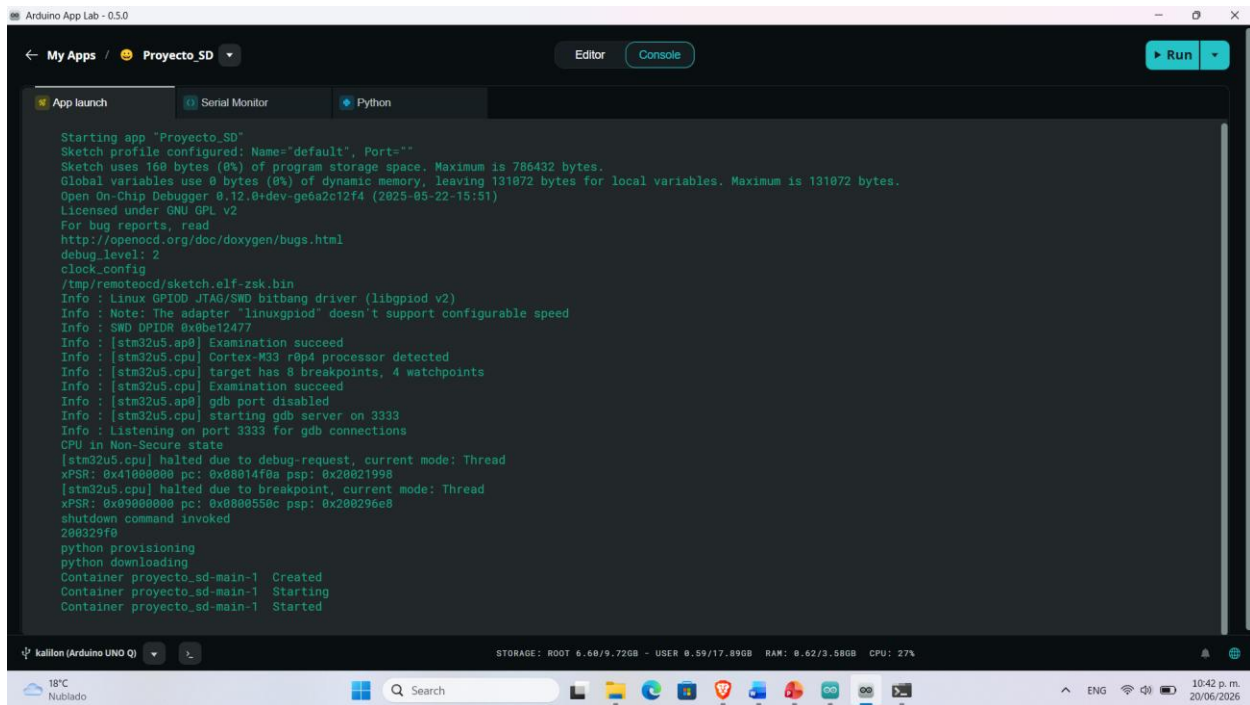
Pero también y para esta ocasión creamos un nuevo proyecto y ahí es donde pegamos nuestro código



En la carpeta de python/main.py

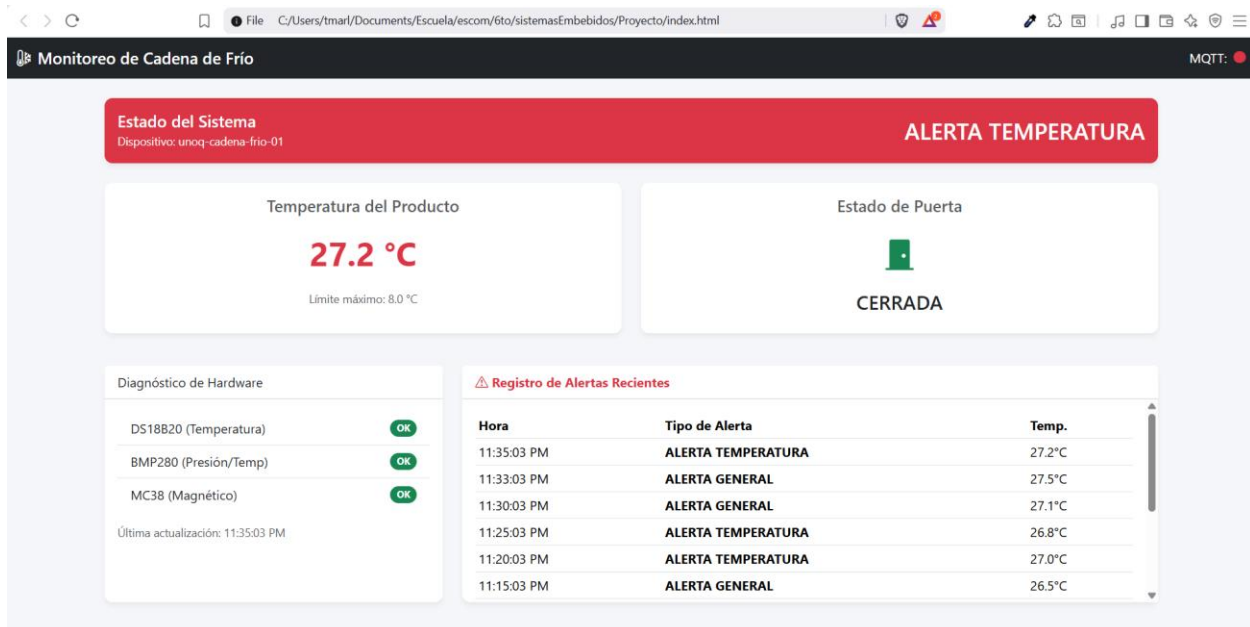


Y seleccionamos la opción de “Run”, ahí mismo en el IDE se nos mostrara el progreso de la carga y ejecución del proyecto.



Funcionamiento

Finalmente, al tener ejecutando el sistema, se puede observar que funciona correctamente, el sistema muestra los cambios en la temperatura y en el sensor de la puerta, se puede observar en la siguiente imagen.



Conclusiones

Para el desarrollo de este proyecto, me tomo mas tiempo del que esperaba, creí que iba a estar sencillo, sin embargo, no lo fue, se tuvo que armar el circuito, y además se tiene que considerar la curva de aprendizaje que se tomo para poder comprender como es que funciona la Arduino Uno Q, y como es que se iba a crear el proyecto para poder realizar lo que se pidió, es por esto que me agrado el proyecto, ya que pude comprender el como se diferencia el MCU y el MPU desde la vista de la arquitectura, además el como esta a su vez se conecta con MQTT, y como se tipo estandariza o se hace el contrato de que información se debe compartir entre las entidades. Además no hubo suficiente tiempo, pero me hubiera gustado implementar este proyecto con alguna nube, por ejemplo la de AWS, para que el proyecto tuviera una escala mayor, para que los listeners, pudieran escuchar desde una red que no fuera local, es decir, se pudiera visualizar toda esta información desde otro lado y además poner en marcha los actuadores, para que así se pudiera simular un entorno mas real de como se usa esta arquitectura, que al menos para este proyecto hubiera tenido un gran impacto, al tener monitoreado las variables de temperatura en un contenedor se pudiera agregar sensores como un GPS, un MPU5060 para monitorear el movimiento y localización, y quizá agregar un modulo que sea de tipo Lora para poder hacer comunicaciones en una red que tenga un radio de alcance mayor por ejemplo cuando una carga se esta transportando por tierra, o los sensores están lejos de los dispositivos de salida de red, esto seria bueno para un trabajo futuro.